

Aplicações dos Conteúdos de Base da UC Algoritmos e Programação

Resumo individual — Algoritmos e Programação (AP) — Linguagem: Java

1. Variáveis e Tipos de Dados

Variáveis são espaços de memória usados para armazenar valores que podem mudar durante a execução de um programa. Em Java, a linguagem é fortemente tipada, ou seja, todo tipo deve ser declarado explicitamente antes do uso (int, double, boolean, String, entre outros).

Aplicação prática: em um sistema de cadastro de clientes de uma loja online, cada informação do cliente é armazenada em uma variável com tipo apropriado, como no exemplo abaixo.

```
String nome = "Maria Silva";  
double saldoCarteira = 250.75;  
boolean cadastroAtivo = true;
```

2. Estruturas Condicionais (if / else)

As estruturas condicionais permitem que o programa tome decisões, executando diferentes blocos de código dependendo se uma condição é verdadeira ou falsa.

Aplicação prática: em um sistema de login de aplicativo bancário, o programa verifica se a senha digitada corresponde à senha cadastrada.

```
if (senhaDigitada.equals(senhaCadastrada)) {  
    System.out.println("Acesso liberado.");  
} else {  
    System.out.println("Senha incorreta.");  
}
```

3. Estruturas de Repetição (for / while)

Laços como for e while permitem repetir um bloco de instruções várias vezes, evitando reescrever o mesmo código manualmente.

Aplicação prática: em um sistema de folha de pagamento, um laço percorre a lista de funcionários para calcular o salário líquido de cada um.

```
for (Funcionario f : listaFuncionarios) {  
    double liquido = f.getSalarioBruto() - f.getDescontos();  
    System.out.println(f.getNome() + ": R$ " + liquido);  
}
```

4. Métodos (Funções) e Modularização

Em Java, blocos de código reutilizáveis são chamados de métodos. Eles recebem parâmetros de entrada e podem retornar um resultado, permitindo dividir um problema grande em partes menores.

Aplicação prática: em um aplicativo de delivery, um método calcula o valor do frete e é reutilizado em todos os pedidos.

```
public static double calcularFrete(double distancia, double peso) {  
    return (distancia * 0.8) + (peso * 1.5);  
}  
  
// uso do método:  
double frete = calcularFrete(12.5, 3.0);
```

5. Vetores, Listas e Estruturas de Dados

Vetores (arrays) e coleções como `ArrayList` permitem armazenar múltiplos valores relacionados sob um único nome, acessíveis por índice.

Aplicação prática: em um sistema de controle de estoque, os produtos de um supermercado podem ser armazenados em uma lista para facilitar consultas e relatórios.

```
ArrayList<String> estoque = new ArrayList<>();  
estoque.add("Arroz 5kg");  
estoque.add("Feijão 1kg");  
  
for (String produto : estoque) {  
    System.out.println(produto);  
}
```

6. Recursividade

A recursividade ocorre quando um método chama a si mesmo para resolver um problema, dividindo-o em subproblemas menores até atingir um caso base.

Aplicação prática: exploradores de arquivos usam algoritmos recursivos para navegar por pastas que contêm outras pastas dentro delas. Um exemplo mais simples e clássico é o cálculo de fatorial:

```
public static int fatorial(int n) {  
    if (n <= 1) {  
        return 1; // caso base  
    }  
    return n * fatorial(n - 1); // chamada recursiva  
}
```